

TASK 3 — DATA LOGGER: SENSOR → SD CARD

Arduino UNO + LM35 + LDR + SD Card Module + DS3231/DS1307 RTC

1. Goal

Collect sensor data such as temperature and light level and save timestamped entries to an SD card in CSV format. The project below uses an LM35 temperature sensor and an LDR as example sensors, with an RTC providing date and time.

2. Hardware Requirements

- Arduino UNO
- Micro SD card module + microSD card
- LM35 temperature sensor
- LDR (photoresistor) + 10 kΩ resistor
- DS3231 or DS1307 RTC module
- Breadboard and jumper wires
- USB cable/power supply

3. Software Requirements

- Arduino IDE
- SD.h — SD card interface
- SPI.h — SPI communication
- RTCLib — RTC support (install from Library Manager)

4. Wiring Connections

Device	Pin	Arduino UNO
LM35	VCC	5V
LM35	OUT	A0
LM35	GND	GND
LDR voltage divider	Analog output	A1
LDR divider	5V / GND	5V / GND
SD module	CS	D10
SD module	MOSI	D11
SD module	MISO	D12
SD module	SCK	D13
SD module	VCC/GND	Module-rated supply / GND
RTC	SDA	A4
RTC	SCL	A5
RTC	VCC/GND	Module-rated supply / GND

Power note: Check the voltage requirements of your particular SD module. Many breakout modules include regulation/level shifting, but bare 3.3 V SD cards must not be connected directly to 5 V logic. Use a common ground between the Arduino and modules.

5. Arduino Program

```
// Task 3: Data Logger - Sensor to SD Card
// Sensors: LM35 (A0) and LDR (A1)
// RTC: DS3231/DS1307 using RTCLib
// SD card: SPI, CS on D10

#include <SPI.h>
#include <SD.h>
#include <Wire.h>
#include <RTCLib.h>

const int SD_CS = 10;
const int TEMP_PIN = A0;
const int LIGHT_PIN = A1;

RTC_DS3231 rtc;    // For DS1307, use RTC_DS1307 rtc;

unsigned long sampleNumber = 0;
const unsigned long SAMPLE_INTERVAL = 1000; // 1 second

void setup()
{
  Serial.begin(9600);
  Wire.begin();

  // Start RTC
  if (!rtc.begin())
  {
    Serial.println("RTC not found!");
    while (1);
  }

  // Uncomment once when setting a new RTC:
  // rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));

  // Start SD card
  if (!SD.begin(SD_CS))
  {
    Serial.println("SD card initialization failed!");
    while (1);
  }

  // Create CSV header if the file is new/empty
  File dataFile = SD.open("DATA.CSV", FILE_WRITE);

  if (dataFile)
  {
    if (dataFile.size() == 0)
    {
      dataFile.println("Sample,Date,Time,Temperature_C,Light_ADC");
    }
    dataFile.close();
  }
  else
  {
    Serial.println("Cannot open DATA.CSV");
    while (1);
  }

  Serial.println("Data logger started.");
}

void loop()
{
  static unsigned long lastSample = 0;

  if (millis() - lastSample >= SAMPLE_INTERVAL)
  {
    lastSample = millis();
    sampleNumber++;

    // Read sensors
    int tempRaw = analogRead(TEMP_PIN);
    int lightRaw = analogRead(LIGHT_PIN);

    // Convert LM35 ADC value to temperature
    float voltage = tempRaw * (5.0 / 1023.0);
    float temperatureC = voltage * 100.0;

    // Read date and time
    DateTime now = rtc.now();

    // Open CSV file and append one record
```

```

File dataFile = SD.open("DATA.CSV", FILE_WRITE);

if (dataFile)
{
    dataFile.print(sampleNumber);
    dataFile.print(",");

    dataFile.print(now.year());
    dataFile.print("-");
    if (now.month() < 10) dataFile.print("0");
    dataFile.print(now.month());
    dataFile.print("-");
    if (now.day() < 10) dataFile.print("0");
    dataFile.print(now.day());
    dataFile.print(",");

    if (now.hour() < 10) dataFile.print("0");
    dataFile.print(now.hour());
    dataFile.print(":");
    if (now.minute() < 10) dataFile.print("0");
    dataFile.print(now.minute());
    dataFile.print(":");
    if (now.second() < 10) dataFile.print("0");
    dataFile.print(now.second());
    dataFile.print(",");

    dataFile.print(temperatureC, 1);
    dataFile.print(",");
    dataFile.println(lightRaw);

    dataFile.close();

    Serial.print("Saved sample ");
    Serial.println(sampleNumber);
}
else
{
    Serial.println("Error opening DATA.CSV");
}

// Stop after at least 100 samples
if (sampleNumber >= 100)
{
    Serial.println("100 samples completed.");
    while (1);
}
}
}

```

6. Code Explanation

SPI.h: Enables SPI communication between Arduino and the SD card module.

SD.h: Creates/opens the CSV file and appends sensor readings to it.

Wire.h: Provides I2C communication used by the RTC.

RTCLib: Reads date and time from the DS3231/DS1307 RTC.

LM35: The analog value from A0 is converted to voltage and then to temperature using the nominal 10 mV/°C sensitivity.

LDR: The voltage-divider output is read at A1. The saved Light_ADC value is an ADC reading from 0 to 1023; its exact relation to illuminance depends on the LDR and resistor circuit.

CSV logging: Each sample is appended as one comma-separated row containing sample number, date, time, temperature and light ADC value.

7. Sampling Rate

The program uses a 1-second sampling interval. Therefore, the nominal sampling rate is 1 sample per second (1 Hz). The 100-sample requirement takes approximately 100 seconds, plus small processing and SD-write overhead.

Sampling interval = 1000 ms

Sampling rate = 1 / 1 s = 1 Hz

A 1-second rate is suitable for slowly changing environmental quantities such as room temperature and light. A faster rate is possible, but unnecessary frequent SD writes can increase power use and reduce the practical lifetime of flash storage. For lower power, samples can be buffered and written in batches or the Arduino can sleep between samples.

8. Power Considerations

The SD card can create short current peaks during write operations. Use a stable power source, short wiring, and adequate decoupling where appropriate. Keep sensor and module grounds common. Follow the voltage specification of the SD module and RTC board. For battery operation, reduce the sampling frequency or batch writes when continuous high-frequency logging is not required.

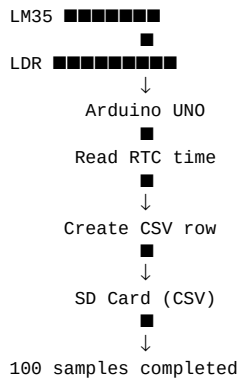
9. Example CSV File

```
Sample,Date,Time,Temperature_C,Light_ADC
1,2026-09-25,10:00:01,25.4,612
2,2026-09-25,10:00:02,25.5,620
3,2026-09-25,10:00:03,25.5,625
4,2026-09-25,10:00:04,25.6,631
...
100,2026-09-25,10:01:40,26.2,705
```

10. Expected Serial Monitor Output

```
Data logger started.
Saved sample 1
Saved sample 2
Saved sample 3
...
Saved sample 100
100 samples completed.
```

11. Working Flow



12. Test Procedure

1. Format the microSD card with a compatible filesystem and insert it into the SD module.
2. Wire the Arduino, sensors, RTC and SD module according to the table.
3. Install SD, SPI and RTCLib libraries as needed.
4. Upload the program to the Arduino UNO.
5. Open Serial Monitor at 9600 baud.
6. Allow the logger to collect at least 100 samples.
7. Remove the SD card safely after logging has stopped and open DATA.CSV on a computer.
8. Verify that the CSV contains timestamped temperature and light readings.

13. Troubleshooting

- **SD initialization failed:** Check CS, SPI wiring, card formatting, module voltage requirements and common GND.
- **RTC not found:** Check SDA/SCL connections and module power.
- **Wrong time:** Set the RTC time once using `rtc.adjust()`, upload, then comment that line and upload again so the clock is not reset every boot.
- **Temperature seems wrong:** Verify LM35 pin orientation and the ADC reference assumption.
- **Light value is reversed:** Depending on which side of the voltage divider contains the LDR, more light may produce a higher or lower ADC value.

14. Deliverables Checklist

- CSV file containing at least 100 samples
- Arduino code
- Wiring diagram/connections
- Explanation of sampling rate
- Explanation of power considerations
- Demonstration of the saved CSV data

15. Result

The data logger collects temperature and light sensor readings, attaches a real-time timestamp from the RTC, and stores the readings as comma-separated records on an SD card. The program is configured to produce at least 100 samples at a nominal 1 Hz sampling rate.

Note

If you use a DS1307 instead of a DS3231, change **RTC_DS3231 rtc;** to **RTC_DS1307 rtc;**. The exact wiring and voltage requirements should be checked against the particular SD and RTC modules being used.